

Project Report: Design and Implementation of a Simple Two-Pass Compiler

1. System Design and Architecture

The project is organized as a two-pass compiler implemented manually in Java.

Pass 1 is the lexer. It reads source code, skips whitespace and comments, classifies lexemes into tokens, reports lexical errors, and builds a symbol table for declared identifiers.

Pass 2 contains the parser and semantic analyzer. The parser validates the token stream with a recursive-descent strategy and builds an AST. The semantic analyzer checks declaration order, duplicate declarations, and type compatibility in assignments and expressions.

The Swing GUI connects these phases and displays the token stream, symbol table, AST, line-to-token mapping, and all compiler errors.

2. Source Language Grammar

```
<program>          ::= <statement>*

<statement>        ::= <declaration>
                       | <assignment>
                       | <if_statement>
                       | <while_statement>
                       | <print_statement>

<declaration>      ::= <type> IDENTIFIER ";"
<type>             ::= "int" | "float"

<assignment>       ::= IDENTIFIER "=" <expression> ";"

<if_statement>      ::= "if" "(" <expression> ")" <block> [ "else" <block> ]
<while_statement>   ::= "while" "(" <expression> ")" <block>
<print_statement>  ::= "print" "(" <expression> ")" ";"
```

```

<block> ::= "{" <statement>* "}"

<expression> ::= <logical_or>
<logical_or> ::= <logical_and> ( "||" <logical_and> )*
<logical_and> ::= <equality> ( "&&" <equality> )*
<equality> ::= <comparison> ( ( "==" | "!=" ) <comparison> )*
<comparison> ::= <term> ( ( "<" | ">" | "<=" | ">=" ) <term> )*
<term> ::= <factor> ( ( "+" | "-" ) <factor> )*
<factor> ::= <unary> ( ( "*" | "/" ) <unary> )*
<unary> ::= ( "!" | "-" | "+" ) <unary> | <primary>
<primary> ::= IDENTIFIER
              | INTEGER_LITERAL
              | FLOAT_LITERAL
              | STRING_LITERAL
              | "(" <expression> ")"

```

3. Lexer Implementation

The lexer is implemented in `src/lexer/Lexer.java`. It scans the source text character by character and generates token objects containing token type, value, and line number.

It recognizes:

- keywords: `int`, `float`, `if`, `else`, `while`, `print`
- identifiers
- integer and floating-point literals
- string literals
- arithmetic, comparison, assignment, and logical operators
- delimiters such as semicolon, parentheses, braces, and comma

The lexer skips whitespace, `//` single-line comments, and `/* ... */` block comments. It reports malformed numbers, invalid characters, unclosed strings, and unclosed block comments as lexical errors.

4. Parser and Semantic Analyzer

The parser is implemented in `src/parser/Parser.java`. It is a hand-written recursive-descent parser. Statements are parsed first, then expressions are parsed using precedence levels. This ensures that multiplication and division

bind more tightly than addition and subtraction, and comparison/logical operators are parsed at lower precedence levels.

The parser builds an AST with `ASTNode`. Each statement and expression operator appears as a node in the tree.

The semantic analyzer is implemented in `src/semantic/SemanticAnalyzer.java`. It performs:

- duplicate declaration detection
- declaration-before-use checking
- assignment type checking
- expression type inference for arithmetic, comparison, and logical operations

The language allows assigning `int` values to `float` variables, but not assigning `float`, `string`, or `boolean` expressions to `int` variables.

5. Symbol Table

The symbol table is implemented with `Symbol` and `SymbolTable` classes. Each symbol stores:

- name
- type
- scope
- memory location

Memory locations start at `1000` and increase by 4 for each declared variable.

The current project uses a global scope, which is sufficient for the required language subset.

6. Error Handling Strategy

The compiler reports three categories of errors:

- Lexical errors: invalid characters, malformed numbers, unclosed strings, and unclosed block comments
- Syntax errors: unexpected tokens, missing parentheses/braces/semicolons, and malformed expressions
- Semantic errors: duplicate variables, variables used before declaration, and incompatible assignment/expression types

All errors include line numbers when the offending token is available.

7. User Interface

The GUI is implemented with Java Swing in `src/gui/CompilerGUI.java`.

Main UI actions:

- write source code directly in the editor
- load a source file with `LOAD FILE`
- run the compiler with `ANALYZE`

Main output tabs:

- Tokens
- Symbol Table
- AST
- Errors
- Source Analysis

Before final submission, add screenshots here:

1. Valid program analysis screen
2. Lexical error screen
3. Syntax error screen
4. Semantic error screen

8. Challenges and Solutions

Challenge: Operator precedence cannot be handled correctly by simply reading tokens until a semicolon or closing parenthesis.

Solution: The parser was changed to a recursive-descent expression parser with separate precedence levels for logical, comparison, additive, multiplicative, unary, and primary expressions.

Challenge: Declaration checking must respect source order.

Solution: The semantic analyzer now scans tokens in program order and adds variables to the declared set only when their declaration is reached.

Challenge: Comments should not become operators or identifiers.

Solution: The lexer now explicitly skips `//` and `/* ... */` comments before operator recognition.

9. Group Member Responsibilities

Update this section with the real group member names before submission.

- Member 1: Lexer, token model, lexical error handling, Semantic analyzer, symbol table, type checking
- Member 2: Parser, AST generation, grammar design, GUI, testing, documentation